Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$= 448$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.
Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.
Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.
Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$$

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- **Accuracy:** 59.63%
- **Precision (macro):** 0.6371
- **Recall (macro):** 0.5992
- **F1-score (macro):** 0.6004
- **Confusion Matrix:** (Refer to Figure 5 in Section 9)
- **Classification Report:**

	precision	recall	f1-score	support
airplane	0.49	0.79	0.60	279
automobile	0.82	0.70	0.75	290
bird	0.57	0.37	0.45	318
cat	0.43	0.43	0.43	303
deer	0.47	0.66	0.55	309
dog	0.51	0.60	0.55	315
frog	0.80	0.57	0.66	292
horse	0.60	0.72	0.66	292
ship	0.92	0.49	0.64	295
truck	0.77	0.66	0.71	307
accuracy			0.60	3000
macro avg	0.64	0.60	0.60	3000
weighted avg	0.64	0.60	0.60	3000

9. Mandatory Plots

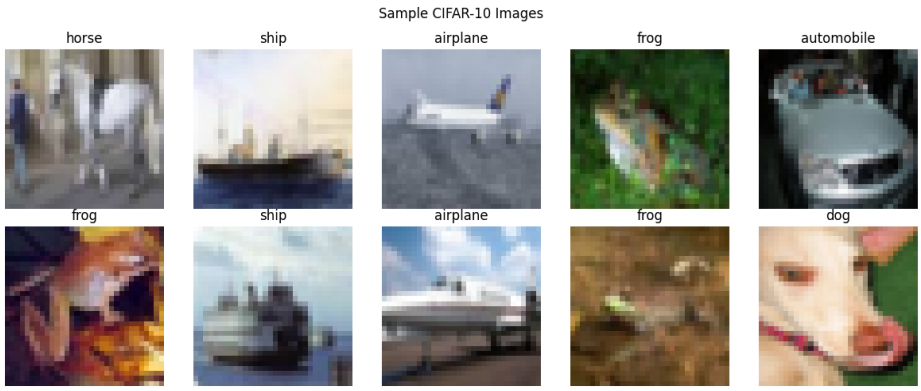

Figure 1: Sample Images

The sample grid displays categories such as horse, ship, airplane, frog, automobile, and dog. This confirms that the dataset loaded correctly and that the labels properly align with the corresponding images before further processing.

Figure 2: Class Distribution

The training subset contains approximately 1500 images per class, indicating an even distribution without major overrepresentation. This balance ensures the model does not become biased toward a majority class, meaning subsequent accuracy differences reflect visual similarities between classes rather than dataset imbalance.

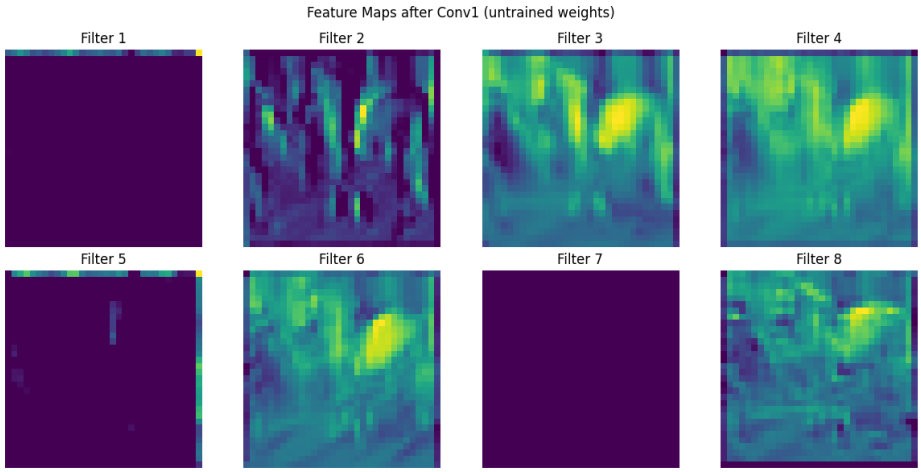

Figure 3: Feature Maps

These feature maps are generated by the first convolutional layer prior to training, showing the reaction of random weights to the input image. Some filters remain dark, indicating they did not detect prominent patterns, while others display bright regions highlighting areas of higher contrast. This behavior is typical for an untrained layer, and these filters will evolve into specific edge or texture detectors during training.

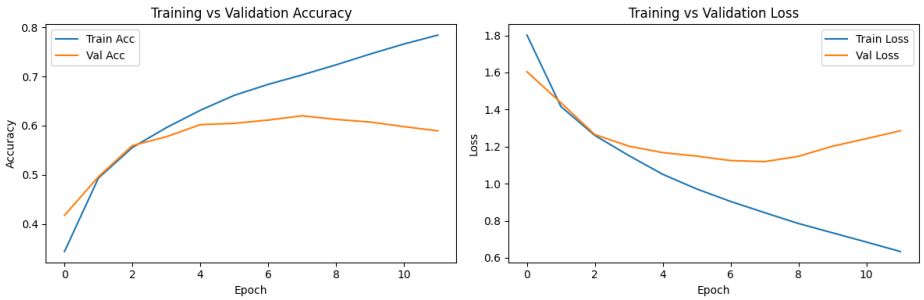

Figure 4: Training and Validation Accuracy/Loss

Training accuracy continuously increases to approximately 78%, whereas validation accuracy plateaus near 60% and slightly declines toward the end. Similarly, training loss steadily decreases, but validation loss reaches a minimum around epoch 6 or 7 before increasing again. This divergence indicates overfitting, where the model memorizes training images rather than learning generalizable patterns, likely due to using a reduced data subset.

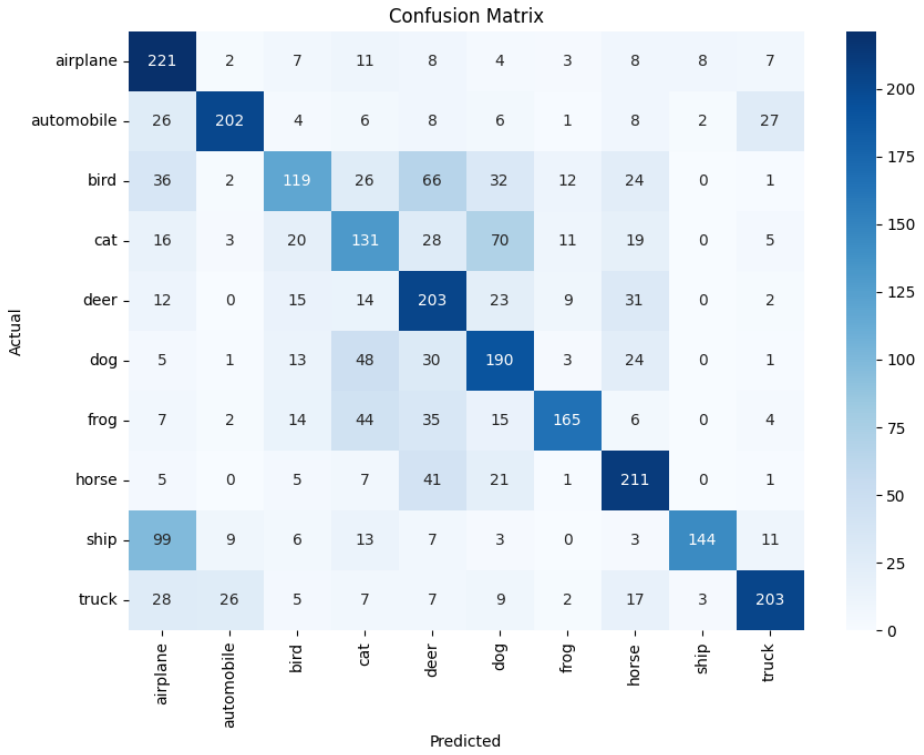

Figure 5: Confusion Matrix

Most classes are correctly predicted along the diagonal, but significant misclassifications occur between airplane and ship, as well as among animal classes such as bird, deer, dog, and frog. Airplane and ship classes share plain backgrounds, and the animal classes share visual similarities at low resolutions, making it difficult for the CNN to cleanly distinguish them.

10. Additional Exercises

1. **Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.**

Output size = 32×32. Since you cannot have half a pixel, it rounds down automatically.

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**
   Total parameters = 1,792. Each filter looks at a 3×3 patch across all 3 colour channels, yielding 27 weights, plus 1 bias, multiplied by 64 filters.
3. **Compare ReLU and Sigmoid activation functions.**
   ReLU is fast and prevents vanishing gradient problems during training by keeping positive values and zeroing negative ones. Sigmoid squashes values between 0 and 1, which leads to vanishing gradients where the network stops learning in earlier layers. ReLU became the standard because it trains faster and works better with deep networks, while Sigmoid remains suitable for the final layer in binary classification.
4. **Replace Max Pooling with Average Pooling and compare the results.**
   Max pooling selects the maximum value in each window, retaining the strongest features. Average pooling calculates the average, which smooths out the features and loses sharpness. In the results, max pooling performed better than average pooling because CIFAR images contain many edges and textures, and max pooling effectively preserves sharp details.
5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**
   Increasing the number of filters allows the model to detect more diverse patterns, which generally improves accuracy. However, more filters require more calculations per step, significantly increasing training time and the number of parameters. On the smaller dataset, the accuracy improvement was marginal, but the increase in computation time was noticeable.

11. Results

- **Training Accuracy:** 78.43%
- **Testing Accuracy:** 59.63%
- **Precision:** 0.6371
- **Recall:** 0.5992
- **F1-score:** 0.6004
- **Number of Parameters:** 282,250

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?**
   Fully connected layers treat every pixel as a separate input, requiring an excessive number of weights for even small images, and fail to capture the spatial relationships between nearby pixels. Convolution addresses this by sliding a small filter over the image, reusing the same weights. This significantly reduces the number of parameters and effectively captures spatial patterns such as edges.
2. **How does stride affect the feature map size?**
   Stride defines the number of pixels the filter shifts at each step. A stride of 1 moves one pixel at a time, resulting in an output close to the original size. A stride of 2 skips pixels, causing the output size to reduce more rapidly. A larger stride produces a smaller feature map and requires less computation.
3. **What is the role of padding?**
   Padding adds extra pixels around the edges of the image prior to convolution. Without padding, corner and edge pixels are processed less frequently than central pixels, leading to information loss and a continuous reduction in spatial dimensions across layers. 'Same' padding maintains the output

size equal to the input size, while 'Valid' padding applies no padding, allowing the dimensions to shrink.

4. **Why is pooling used?**
   Pooling reduces the spatial dimensions of feature maps, which decreases computation and the number of subsequent parameters. It provides translation invariance, enabling the model to detect features even if their exact pixel positions shift slightly. Pooling improves efficiency and enhances the model's robustness to minor spatial variations.
5. **How do feature maps represent image characteristics?**
   Each feature map is generated by a filter scanning the image, highlighting regions where the corresponding pattern—such as an edge, texture, or color gradient—is detected. Early layers capture simple features, while deeper layers combine these to represent more complex shapes related to objects. Feature maps illustrate the specific visual characteristics the model is focusing on at each stage.
6. **Why do CNNs require fewer parameters than MLPs?**
   CNNs require fewer parameters due to parameter sharing and local connectivity. In an MLP, every neuron connects to every pixel, rapidly increasing the weight count. In a CNN, a single filter uses a small, fixed set of weights that slides across the entire image. This approach covers the image with significantly fewer parameters and reduces the risk of overfitting on small datasets.

13. Sources

- **GitHub Repository:** `https://github.com/Saraswathi-Nalamothu/deep-learning_experiments`
- **Colab Notebook:** `https://colab.research.google.com/drive/1aPZcwjGrBsQRoWWiP7CZAyj8MTAgjaWz?usp=sharing`